

# Tarea-Examen 3

## Complejidad Computacional

Berriel Flores José M.

Junio 2026

### Problema 3

#### (a) Cláusulas cortas

(i) **Cláusula corta:**  $C = (\ell_1)$ . Sean las variables auxiliares  $z_1, z_2$  y reemplace  $C$  por la conjunción de cláusulas de exactamente 3 literales:

$$C' = (\ell_1 \vee z_1 \vee z_2) \wedge (\ell_1 \vee z_1 \vee \bar{z}_2) \wedge (\ell_1 \vee \bar{z}_1 \vee z_2) \wedge (\ell_1 \vee \bar{z}_1 \vee \bar{z}_2).$$

- $(\Rightarrow)$  Si  $\ell_1 = 1$ , entonces cada una de las cuatro cláusulas se satisface independientemente del valor de  $z_1, z_2$ .
- $(\Leftarrow)$  Si  $\ell_1 = 0$ , necesitamos satisfacer las cuatro cláusulas. Las cuatro combinaciones posibles de  $(z_1, z_2) \in \{0, 1\}^2$  aparecen en  $C'$  pero no hay manera de satisfacer cada cláusula cuando  $\ell_1 = 0$  y toma exactamente la combinación opuesta de  $z_1, z_2$ . Por lo tanto, ningún valor de  $z_1, z_2$  satisface las cuatro cláusulas simultáneamente cuando  $\ell_1 = 0$ .

Con lo cual,  $C'$  es satisfactible si y sólo si  $\ell_1 = 1$ , es decir, si y sólo si  $C$  es satisfactible.

(ii) **Cláusula corta:**  $C = (\ell_1 \vee \ell_2)$ . Introducimos una variable auxiliar  $z$  y reemplazamos  $C$  por:

$$C' = (\ell_1 \vee \ell_2 \vee z) \wedge (\ell_1 \vee \ell_2 \vee \bar{z}).$$

- $(\Rightarrow)$  Si  $\ell_1 \vee \ell_2 = 1$ , ambas cláusulas son verdaderas independientemente del valor de  $z$ .
- $(\Leftarrow)$  Si  $\ell_1 = \ell_2 = 0$ , entonces  $(\ell_1 \vee \ell_2 \vee z)$  requiere  $z = 1$  y  $(\ell_1 \vee \ell_2 \vee \bar{z})$  requiere  $z = 0$ , lo cual no es posible. Por lo tanto,  $C'$  es insatisfactible cuando  $C$  lo es.

#### (b) Cláusula de 4 literales: $C = (\ell_1 \vee \ell_2 \vee \ell_3 \vee \ell_4)$

Sea la variable auxiliar  $z$  y defina las cláusulas:

$$C_1 = (\ell_1 \vee \ell_2 \vee z), \quad C_2 = (\bar{z} \vee \ell_3 \vee \ell_4).$$

### (c) Correctud del reemplazo

Debemos demostrar que  $C_1 \wedge C_2$  es satisfacible si y sólo si  $C$  lo es.

( $\Rightarrow$ ) **Suponga  $C$  satisfacible.** Sea  $\psi$  una asignación que satisface  $C = (\ell_1 \vee \ell_2 \vee \ell_3 \vee \ell_4)$ . Entonces al menos uno de los literales  $\ell_1, \ell_2, \ell_3, \ell_4$  es verdadero bajo  $\psi$ . Distinguiamos dos casos:

- Si  $\ell_1 = 1$  ó  $\ell_2 = 1$  bajo  $\sigma$ : y  $z = 0$ . Entonces se satisface  $C_1 = (\ell_1 \vee \ell_2 \vee 0)$  (pues  $\ell_1 \vee \ell_2 = 1$ ), y  $C_2 = (1 \vee \ell_3 \vee \ell_4)$  también es satisfecha.
- Si  $\ell_1 = \ell_2 = 0$  bajo  $\psi$ : necesariamente  $\ell_3 = 1$  ó  $\ell_4 = 1$ . Si  $z = 1$ . Entonces  $C_1 = (\ell_1 \vee \ell_2 \vee 1)$  es satisfecha, y  $C_2 = (0 \vee \ell_3 \vee \ell_4)$  es satisfecha porque  $\ell_3 \vee \ell_4 = 1$ .

En ambos casos existe un valor de  $z$  tal que  $C_1 \wedge C_2$  se satisface bajo la asignación  $\psi$ .

( $\Leftarrow$ ) **Suponga  $C_1 \wedge C_2$  satisfacible.** Sea  $\sigma$  una asignación que satisface  $C_1 \wedge C_2$ .

- Si  $\sigma(z) = 0$ : de  $C_1 = (\ell_1 \vee \ell_2 \vee 0)$  se sigue que  $\ell_1 = 1$  ó  $\ell_2 = 1$ , entonces  $C = (\ell_1 \vee \ell_2 \vee \ell_3 \vee \ell_4)$  es verdadera (independientemente del valor de  $z$ ).
- Si  $\sigma(z) = 1$ : de  $C_2 = (0 \vee \ell_3 \vee \ell_4)$  se sigue que  $\ell_3 = 1$  ó  $\ell_4 = 1$  y se satisface  $C$ .

En ambos casos la asignación sobre las variables originales (ignorando  $z$ ) satisface  $C$ .

### (d) Generalización:

Sea  $C = (\ell_1 \vee \ell_2 \vee \dots \vee \ell_k)$  con  $k > 3$ .

(i) **Construcción recursiva.** Defina las variables auxiliares  $z_1, z_2, \dots, z_{k-3}$  y la siguiente conjunción:

$$\begin{aligned} C_1 &= (\ell_1 \vee \ell_2 \vee z_1), \\ C_2 &= (\bar{z}_1 \vee \ell_3 \vee z_2), \\ C_3 &= (\bar{z}_2 \vee \ell_4 \vee z_3), \\ &\vdots \\ C_{k-3} &= (\bar{z}_{k-4} \vee \ell_{k-2} \vee z_{k-3}), \\ C_{k-2} &= (\bar{z}_{k-3} \vee \ell_{k-1} \vee \ell_k). \end{aligned}$$

El reemplazo de  $C$  es  $C_1 \wedge C_2 \wedge \dots \wedge C_{k-2}$ .

(ii) **Variables auxiliares introducidas.** Se introducen exactamente  $k - 3$  variables auxiliares:  $z_1, \dots, z_{k-3}$ .

(iii) **Cláusulas producidas.** La construcción produce exactamente  $k - 2$  cláusulas de tamaño 3.

(iv) **Correctud por inducción. Caso base** ( $k = 4$ ): Es exactamente el inciso (b)/(c), ya demostrado.

**Paso inductivo:** Supongamos que la construcción es correcta para cláusulas de tamaño  $k-1$ . Dada  $C = (\ell_1 \vee \dots \vee \ell_k)$ , la construcción produce  $C_1 = (\ell_1 \vee \ell_2 \vee z_1)$  y el resto equivale a la reducción de  $C' = (\bar{z}_1 \vee \ell_3 \vee \dots \vee \ell_k)$  de tamaño  $k-1$  (con variables auxiliares  $z_2, \dots, z_{k-3}$ ), que por hipótesis inductiva es equisatisfactible con  $C'$ .

- $(\Rightarrow)$  Si  $C$  es satisfactible, algún  $\ell_j = 1$ . Si  $j \in \{1, 2\}$ , y  $z_1 = 0$ :  $C_1$  se satisface y  $C' = (\bar{0} \vee \ell_3 \vee \dots) = (1 \vee \dots)$  también. Si  $j \geq 3$ , y  $z_1 = 1$ :  $C_1 = (\ell_1 \vee \ell_2 \vee 1)$  se satisface y  $C' = (0 \vee \ell_3 \vee \dots \vee \ell_k)$  es verdadera dado que  $\ell_j = 1$  para  $j \geq 3$ . Por hipótesis inductiva, el resto de las cláusulas también se satisface.
- $(\Leftarrow)$  Si la conjunción  $C_1 \wedge \dots \wedge C_{k-2}$  es satisfactible, en particular  $C'$  es satisfactible (por hipótesis inductiva). Si  $z_1 = 1$ , entonces algún  $\ell_j = 1$  para  $j \geq 3$ , y  $C$  es verdadera. Si  $z_1 = 0$ , de  $C_1$  se sigue  $\ell_1 = 1$  ó  $\ell_2 = 1$ , y  $C$  también se satisface.

## (e) La reducción $f$

Sea  $\varphi$  una fórmula FNC con  $m$  cláusulas y longitud total  $s = |\varphi|$ .

(i) **Tamaño de  $\psi = f(\varphi)$ .** Cada cláusula  $C_i$  de longitud  $k_i$  produce a lo más  $k_i - 2$  cláusulas de tamaño 3 (con los casos  $k_i = 1$  y  $k_i = 2$  considerados en la construcción con variables auxiliares). El número total de cláusulas en  $\psi$  es:

$$\sum_{i=1}^m (k_i - 2) \leq \sum_{i=1}^m k_i \leq s.$$

Cada nueva cláusula tiene 3 literales, entonces  $|\psi| = O(s)$ . En particular,  $|\psi| \leq 3s$ .

(ii) **Computabilidad en tiempo polinomial.** Para cada cláusula  $C_i$  de longitud  $k_i$ , por el inciso (d) se introducen  $k_i - 3$  variables nuevas y define  $k_i - 2$  cláusulas en tiempo  $O(k_i)$ . Sumando sobre todas las cláusulas:

$$T(f, \varphi) = O\left(\sum_{i=1}^m k_i\right) = O(s).$$

Luego  $f$  es computable en tiempo  $O(s)$ , que es polinomial en  $|\varphi|$ .

(iii) **Conclusión:  $\text{SAT} \leq_p \text{3SAT}$ .** Para toda fórmula  $\varphi$  en FNC:

$$\varphi \in \text{SAT} \iff f(\varphi) \in \text{3SAT},$$

Dado que para toda  $\varphi \in \text{SAT}$  puede encontrarse una  $f$  computable en tiempo polinomial tal que  $f(\varphi) \in \text{3SAT}$ , e inversamente ocurre la satisfactibilidad. Por definición, SAT es Karp reducible a 3SAT. ■

## Problema 4

### (a) P.D. 3SAT $\in$ NP

Dada una fórmula 3-FCN  $\varphi$  con variables  $x_1, \dots, x_n$ , el testigo es una asignación  $\sigma : \{x_1, \dots, x_n\} \rightarrow \{0, 1\}$ .

El verificador recibe  $(\varphi, \sigma)$  y:

1. Evalúa cada cláusula de  $\varphi$  bajo  $\sigma$ .
2. Acepta si y sólo si todas las cláusulas son verdaderas.

Nótese que  $\varphi$  tiene  $m$  cláusulas de a lo sumo 3 literales cada una y evaluar todas las cláusulas toma tiempo  $O(m) = O(|\varphi|)$ , que es polinomial.

Si  $\varphi \in 3SAT$ , existe  $\sigma$  que la satisface; el verificador acepta. Si  $\varphi \notin 3SAT$ , ninguna asignación satisface  $\varphi$ ; el verificador rechaza para todo  $\sigma$ .

Por tanto,  $3SAT \in \mathbf{NP}$ .

### (b) P.D. 3SAT es NP-difícil

Como SAT es NP-difícil,  $\forall L \in \mathbf{NP}$  sucede que  $L \leq_p SAT$ . Además ya se demostró en el anterior problema que  $SAT \leq_p 3SAT$ .

Sea  $L \in \mathbf{NP}$ . Entonces:

$$L \leq_p SAT \quad \text{y} \quad SAT \leq_p 3SAT.$$

Sean  $g$  la reducción de  $L$  a SAT y  $f$  la reducción de SAT a 3SAT, ambas computables en tiempo polinomial. Defina  $h = f \circ g$ , es decir,  $h(x) = f(g(x))$ .

Entonces:

$$x \in L \iff g(x) \in SAT \iff f(g(x)) \in 3SAT$$

Si  $g$  se ejecuta en tiempo  $p_1(|x|)$  y  $f$  se ejecuta en tiempo  $p_2$ , entonces  $h$  corre en tiempo  $p_1(|x|) + p_2(p_1(|x|))$ , que es polinomial en  $|x|$ .

Por lo tanto  $L \leq_p 3SAT$  para todo  $L \in \mathbf{NP}$ , i.e., 3SAT es NP-difícil.

### (c) Conclusión

Por definición un problema es NP-completo si pertenece a  $\mathbf{NP}$  y es NP-difícil.

- Por el inciso (a):  $3SAT \in \mathbf{NP}$ .
- Por el inciso (b): 3SAT es NP-difícil.

Se puede concluir, por definición, 3SAT es **NP-completo**. ■

## Problema 5

Sea  $D$  un algoritmo polinomial para SAT y sea  $\varphi$  una fórmula FNC satisfactible sobre variables  $x_1, \dots, x_n$ . Notación: Se denota como  $\varphi[x_i \leftarrow b]$  a la fórmula obtenida al sustituir  $x_i$  por  $b \in \{0, 1\}$  y simplificar (eliminar cláusulas satisfechas y literales falsos).

(a)  $\varphi[x_1 \leftarrow 0] \in \text{SAT}$  ó  $\varphi[x_1 \leftarrow 1] \in \text{SAT}$

Como  $\varphi \in \text{SAT}$ , existe  $v = (x_1, x_2, \dots, x_n)$  asignación, tal que  $\varphi(v) = 1$ .

- Si  $x_1 = 0$ : entonces  $v$  restringida a  $x_2, \dots, x_n$  satisface  $\varphi[x_1 \leftarrow 0]$ . Por lo tanto  $\varphi[x_1 \leftarrow 0] \in \text{SAT}$ .
- Si  $x_1 = 1$ : la asignación  $v$  restringida a  $x_2, \dots, x_n$  satisface  $\varphi[x_1 \leftarrow 1]$ . Luego  $\varphi[x_1 \leftarrow 1] \in \text{SAT}$ .

En cualquier caso, al menos una de las dos fórmulas se satisface. ■

(b) **Algoritmo y correctud**

**Algoritmo** ENCUENTRAASIGNACIÓN( $\varphi, D$ )

**Entrada:**  $\varphi$  sobre  $x_1, \dots, x_n$ ; algoritmo  $D$  para SAT.

**Salida:** asignación  $(x_1, \dots, x_n)$  que satisface  $\varphi$ .

1. **for**  $i = 1$  **hasta**  $n$ :
  - a. Calcular  $\varphi_0 \leftarrow \varphi[x_i \leftarrow 0]$ .
  - b. **Si**  $D(\varphi_0) = 1$  **entonces** fijar  $x_i = 0$ , actualizar  $\varphi \leftarrow \varphi_0$ .
  - c. **En otro caso** fijar  $x_i = 1$ , actualizar  $\varphi \leftarrow \varphi[x_i \leftarrow 1]$ .
2. **Devolver**  $(x_1, x_2, \dots, x_n)$ .

**Correctud** Demostración por inducción sobre  $i$ .

*Base* ( $i = 1$ ):  $\varphi$  es satisfactible por hipótesis.

*Paso inductivo*: Suponga que  $\varphi$  es satisfactible para  $x_{i-1}$ . En la iteración  $i$ :

- El algoritmo consulta  $D(\varphi[x_i \leftarrow 0])$ .
- Por el inciso (a) aplicado a  $\varphi$  (con primera variable libre  $x_i$ ), al menos una de  $\varphi[x_i \leftarrow 0]$  o  $\varphi[x_i \leftarrow 1]$  es satisfactible.
- Si  $D(\varphi[x_i \leftarrow 0]) = 1$ , el algoritmo fija  $x_i = 0$  y la nueva  $\varphi$  es satisfactible.
- Si  $D(\varphi[x_i \leftarrow 0]) = 0$ , entonces  $\varphi[x_i \leftarrow 0] \notin \text{SAT}$ ; por (a),  $\varphi[x_i \leftarrow 1] \in \text{SAT}$ , con lo cual el algoritmo fija  $x_i = 1$  y la nueva  $\varphi$  es satisfactible.

Al finalizar el ciclo (después de fijar  $x_1, \dots, x_n$ ), la fórmula  $\varphi$  sigue siendo satisfactible, por lo que  $(x_1, \dots, x_n)$  satisface la fórmula original. ■

### (c) Número de llamadas a $D$ y tiempo polinomial

El ciclo tiene exactamente  $n$  iteraciones y en cada iteración se realiza **exactamente una** llamada a  $D$  (para  $\varphi[x_i \leftarrow 0]$ ). Por tanto, el algoritmo realiza en total  $n$  llamadas a  $D$ .

Sea  $s = |\varphi|$ . Cada sustitución y simplificación toma tiempo  $O(s)$ . Como  $D$  es polinomial y  $|\varphi[x_i \leftarrow b]| \leq s$ , entonces cada llamada a  $D$  toma tiempo  $p(s)$  para algún polinomio  $p$ . El tiempo total es:

$$T(n) = n \cdot (O(s) + p(s)) \leq s \cdot (O(s) + p(s)) = O(s \cdot p(s)),$$

que es polinomial en  $s = |\varphi|$ . Por tanto, el algoritmo corre en tiempo polinomial. ■

### (d) Extensión a cualquier $L \in \mathbf{NP}$

Sea  $L \in \mathbf{NP}$  con reducción de Cook-Levin  $x \mapsto \varphi_x$ , donde:

- $\varphi_x$  es una fórmula FNC computable en tiempo polinomial en  $|x|$ .
- $x \in L \iff \varphi_x \in \text{SAT}$ .
- Por el teorema de Cook-Levin, existen un grupo de variables que codifican a un testigo  $u$  de  $x \in L$ .

#### **Algoritmo** ENCuentraTestigo( $x$ )

1. Calcular  $\varphi_x$  (en tiempo polinomial por la reducción de Cook-Levin).
2. Si  $D(\varphi_x) = 0$ , rechazar ( $x \notin L$ ).
3. Aplicar ENCuentraAsignación( $\varphi_x, D$ ) para obtener una asignación satisfactoria  $(b_1, \dots, b_N)$  de  $\varphi_x$ .
4. Extraer de  $(b_1, \dots, b_N)$  los bits correspondientes a las variables del testigo  $u$  (según la codificación de Cook-Levin) y devolver  $u$ .

**Correctud.** Si  $x \in L$ , entonces  $\varphi_x \in \text{SAT}$  y ENCuentraAsignación devuelve una asignación que satisface  $\varphi_x$ . Por la estructura de la reducción de Cook-Levin, los bits de testigo en dicha asignación forman un testigo válido  $u$  para  $x \in L$ .

El algoritmo completo corre en tiempo polinomial en  $|x|$ :

- Calcular  $\varphi_x$ : tiempo polinomial en  $|x|$ .
- ENCuentraAsignación: tiempo polinomial en  $|\varphi_x|$ , que es polinomial en  $|x|$  (por la reducción de Cook-Levin).
- Extracción del testigo: tiempo polinomial.

Por lo tanto, suponiendo  $\mathbf{P} = \mathbf{NP}$  (lo que garantiza la existencia de  $D$  polinomial para SAT), se puede tanto, decidir si  $x \in L$ , como encontrar un testigo para  $x \in L$  en tiempo polinomial, para cualquier  $L \in \mathbf{NP}$ . ■